

Lesson 9 — Naive Bayes

You've completed:

1. ML Basics
2. Linear Regression
3. Regression Evaluation
4. Classification
5. Classification Evaluation
6. KNN
7. Decision Trees
8. Random Forest
9. Naive Bayes ← Today

Naive Bayes is another classification algorithm, but its thinking is very different from KNN and Decision Trees.

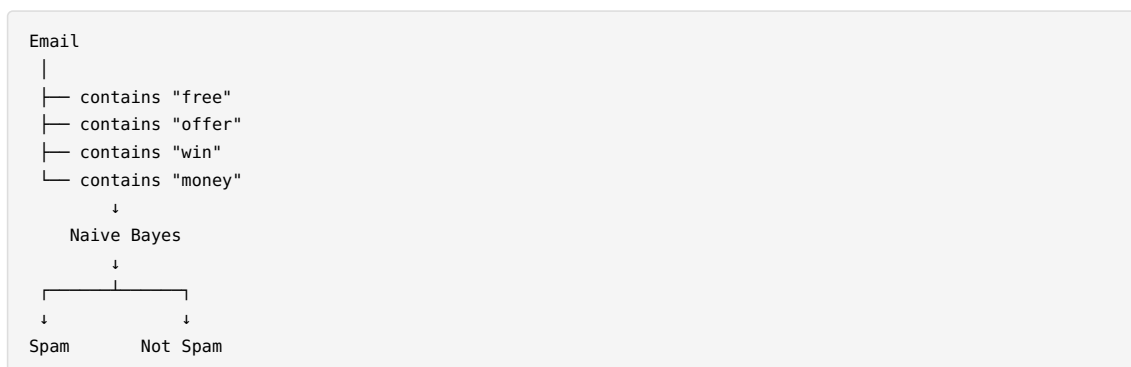
1. What is Naive Bayes?

Naive Bayes is a classification algorithm based on probability.

Its basic question is:

“Given these features, which class is most probable?”

For example, suppose we want to classify an email:



The model calculates probabilities and chooses the most likely class.

2. First understand probability

Suppose you have 100 emails.

80 → Not Spam
20 → Spam

If I randomly select one email, the probability of Spam is:

$$P(\text{Spam}) = 20 / 100 = 0.20$$

And:

$$P(\text{Not Spam}) = 80 / 100 = 0.80$$

So before looking at the email's content, Spam has a probability of 20%.

This initial probability is called the prior probability.

3. Now add evidence

Suppose the email contains:

"Congratulations! You won \$1000!"

We know that words like:

won
money
prize
congratulations

may occur more frequently in spam emails.

So we update our probability.

Conceptually:

```
Before seeing email
  ↓
P(Spam) = 20%
  ↓
Look at email features
  ↓
"won", "prize", "money"
  ↓
Update probability
  ↓
P(Spam | features) = high
  ↓
Spam
```

This idea comes from Bayes' theorem.

4. Bayes' Theorem ☆

The basic formula is:

$$P(A|B) = [P(B|A) \times P(A)] / P(B)$$

Don't worry if the formula looks scary.

Let's translate it.

P(A | B)

Means:

Probability of A given B

The **|** means "given that".

For example:

$P(\text{Spam} \mid \text{"free"})$

means:

Probability that an email is Spam given that it contains the word "free".

5. The four important terms

Bayes' theorem:

$$P(A \mid B) = \frac{P(B \mid A) \times P(A)}{P(B)}$$

$P(A \mid B)$

Posterior probability

Probability of A after seeing B.

$P(B \mid A)$

Likelihood

How likely is B if A is true?

$P(A)$

Prior probability

Probability of A before seeing the new evidence.

$P(B)$

Evidence

Overall probability of B.

6. Simple real-world example

Suppose:

Disease → 1% of people
Healthy → 99%

So:

$$P(\text{Disease}) = 0.01$$

Now suppose a test is positive.

The test has:

$$P(\text{Positive} \mid \text{Disease}) = 0.90$$

That means:

If someone has the disease, the test is positive 90% of the time.

But we also need to know:

$P(\text{Positive} \mid \text{Healthy})$

Suppose:

$P(\text{Positive} \mid \text{Healthy}) = 0.05$

Now Bayes' theorem helps us calculate:

$P(\text{Disease} \mid \text{Positive})$

Notice the difference:

$P(\text{Positive} \mid \text{Disease})$

is NOT the same as:

$P(\text{Disease} \mid \text{Positive})$

This distinction is extremely important.

7. Why is it called “Naive” Bayes?

This is one of the most important concepts.

Naive Bayes makes a strong simplifying assumption:

It assumes that features are conditionally independent given the class.

In simpler language:

Suppose an email contains:

free
money
offer

Naive Bayes treats these features as if their contributions can be considered independently once we know whether the email is spam.

Real-world features may actually be related.

For example:

"free"
⚡
"offer"

may be related.

But Naive Bayes makes the simplifying assumption anyway.

That's why it is called Naive.

8. Why can a “naive” model work well?

Because the assumption simplifies the problem enormously.

Instead of trying to model every possible relationship between features, it calculates their individual contributions.

```
Feature 1 → probability
Feature 2 → probability
Feature 3 → probability
Feature 4 → probability
      ↓
Combine
      ↓
Class probability
```

Despite the simplifying assumption, Naive Bayes can work surprisingly well, especially for certain types of data.

9. Naive Bayes for Spam Detection

This is one of its classic applications.

Suppose training data contains:

```
Email 1 → "free money offer" → Spam
Email 2 → "meeting tomorrow" → Not Spam
Email 3 → "win free prize" → Spam
Email 4 → "project meeting" → Not Spam
```

The model learns associations between words and classes.

Then a new email arrives:

```
"free prize"
```

Naive Bayes calculates something conceptually like:

```
P(Spam | free, prize)

vs

P(Not Spam | free, prize)
```

Whichever probability is higher becomes the prediction.

10. Types of Naive Bayes

Scikit-learn provides several versions.

The most important ones are:

1. Gaussian Naive Bayes

```
GaussianNB()
```

Useful for continuous numerical features.

Example:

```
Age
Height
Temperature
CPU usage
Latency
```

2. Multinomial Naive Bayes

```
MultinomialNB()
```

Very commonly used for:

- text classification
- word counts
- document classification

For example:

```
Spam detection
Sentiment analysis
News classification
```

3. Bernoulli Naive Bayes

```
BernoulliNB()
```

Useful when features are binary:

```
0 / 1
False / True
No / Yes
Absent / Present
```

11. Let's use Gaussian Naive Bayes

We'll start with a simple numerical classification problem.

Use the familiar Breast Cancer dataset.

Import:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
```

12. Load the dataset

```
data = load_breast_cancer()

X = data.data
y = data.target
```

Remember:

X → Features
y → Target

13. Split the data

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

So:

80% → training
20% → testing

14. Create the Naive Bayes model

```
model = GaussianNB()
```

This creates a Gaussian Naive Bayes classifier.

15. Train

```
model.fit(X_train, y_train)
```

The model learns probability distributions from the training data.

16. Predict

```
predictions = model.predict(X_test)
```

Now it predicts the class of each test example.

17. Evaluate

```
accuracy = accuracy_score(y_test, predictions)

print("Accuracy:", accuracy)
```

18. Complete code ☆

Run this:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
```

```

# Load dataset
data = load_breast_cancer()

# Features and target
X = data.data
y = data.target

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Create model
model = GaussianNB()

# Train
model.fit(X_train, y_train)

# Predict
predictions = model.predict(X_test)

# Evaluate
accuracy = accuracy_score(y_test, predictions)

print("Accuracy:", accuracy)

```

19. Naive Bayes can also give probabilities

Just like Logistic Regression and Random Forest, we can ask for probabilities.

Use:

```

probabilities = model.predict_proba(X_test)

print(probabilities[:5])

```

You might see:

```

[[0.01 0.99]
 [0.98 0.02]
 [0.03 0.97]
 [0.95 0.05]
 [0.02 0.98]]

```

For one sample:

```
[0.01, 0.99]
```

could mean:

```

Class 0 → 1%
Class 1 → 99%

```

The exact interpretation depends on:

```
model.classes_
```

You can check:

```
print(model.classes_)
```

20. predict() vs predict_proba()

You've now seen this multiple times.

predict()

Gives the final class.

```
model.predict(X_test)
```

Example:

```
[1, 0, 1, 1, 0]
```

predict_proba()

Gives probabilities.

```
model.predict_proba(X_test)
```

Example:

```
[0.10, 0.90]
```

Think:

```
predict()
  ↓
Final decision

predict_proba()
  ↓
How confident/probable are the classes?
```

21. Naive Bayes vs the models you've learned

Now compare the major classification algorithms:

Algorithm	Main idea
Logistic Regression	Learn mathematical relationship
KNN	Look at nearby examples
Decision Tree	Ask learned questions
Random Forest	Combine many decision trees
Naive Bayes	Calculate probabilities

This is an important ML skill:

Don't just memorize algorithms. Understand how each model thinks.

22. When is Naive Bayes especially useful?

Naive Bayes is particularly famous for:

✉ Spam detection

```
Email
↓
Words
↓
Naive Bayes
↓
Spam / Not Spam
```

📰 Text classification

```
Article
↓
Words
↓
Naive Bayes
↓
Sports / Politics / Technology
```

😊 Sentiment analysis

```
"I love this product!"
↓
Naive Bayes
↓
Positive
```

📄 Document classification

```
Document
↓
Features
↓
Naive Bayes
↓
Category
```

23. Naive Bayes and SentinelAI

Naive Bayes isn't necessarily the first model I'd choose for your SentinelAI telemetry problem, but understanding it is useful.

Your data might look like:

CPU	Memory	Latency	Error Rate
72%	65%	120ms	0.5%
95%	91%	850ms	7.2%

and the target:

```
Normal
Failure
```

Naive Bayes could theoretically classify these based on learned probability distributions.

But later we'll compare models properly using:

Accuracy
Precision
Recall
F1
ROC-AUC
Cross-validation

and choose a model based on actual performance and the problem's requirements.

24. Important limitation

Naive Bayes assumes feature independence.

Imagine:

CPU usage
↓
Request rate
↓
Latency

These can naturally be related in distributed systems.

For example:

Request rate ↑
↓
CPU ↑
↓
Latency ↑

Naive Bayes' independence assumption may not represent such relationships well.

That's one reason we don't simply say:

"Naive Bayes is the best model."

Different datasets favor different algorithms.

Lesson 9 Summary

Remember these:

★ 1

Naive Bayes is a probabilistic classification algorithm.

★ 2

It is based on Bayes' theorem.

$$P(A|B) = [P(B|A) \times P(A)] / P(B)$$

★ 3

The "naive" assumption is:

Features are conditionally independent given the class.

★ **4**

Common implementations:

```
GaussianNB()  
MultinomialNB()  
BernoulliNB()
```

★ **5**

GaussianNB is commonly suitable for continuous numerical features.

★ **6**

MultinomialNB is particularly useful for text/count data.

★ **7**

`predict()` → final class

```
model.predict(X_test)
```

★ **8**

`predict_proba()` → class probabilities

```
model.predict_proba(X_test)
```

Practice

Try these yourself:

Q1

What is the main idea behind Naive Bayes?

Q2

What does this mean?

```
P(Spam | "free")
```

Q3

Why is it called Naive Bayes?

Q4

Which one would you typically consider for text classification?

```
GaussianNB  
MultinomialNB  
BernoulliNB
```

Q5

What's the difference between:

```
model.predict(X_test)
```

and

```
model.predict_proba(X_test)
```

Q6

If:

```
Class 0 → 20%  
Class 1 → 80%
```

what class would `predict()` normally return?